

第26回 実践：Web ゲームプログラミング5（当たり判定と得点）

- 前回：敵が出て、画面外で消えるようにした
- 今回：弾が敵に当たると消え、得点が増える
- 次回：jQuery で表示し、残機を作る

実習時のフォルダ構成

Web_名前 の中に 26 フォルダを作る。第 25 回の sample フォルダと exercise フォルダを、26 の中にコピーして、続きから作る。

```
Web_名前
├── 26
│   ├── sample          ← 第 25 回の sample をコピーする
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 25 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、sample/main.js に書き足す）
- exercise：自分のシューティングゲーム（演習 1～2、発展）
- 今回は、index.html に得点の表示欄を 1 行追加する。style.css は変更しない。

講義内容

sample/main.js に書き足す。書いたら sample/index.html をブラウザで開いて確認する。

1. 前回の復習

- 敵は、frameCount % 60 === 0 のとき（約 1 秒に 1 回）、乱数で決めた x 座標に出てくる。
- 画面の外に出た弾と敵は、後ろから回す for と splice で、配列から消す。

今回は、弾が敵に当たったら、弾と敵を消して、得点を増やす。得点は、第 20 回と同じ方法で HTML に表示する。

2. 今回の main.js の並び

```
// ===== 準備 =====      ← 得点の表示欄を取り出す 1 行を追加する
// ===== ゲームの状態 ===== ← 得点の変数を追加する
// ===== 自機・弾・敵 =====
// ===== キー入力 =====
// ===== 更新 =====      ← isHit と checkBulletHits を追加し、update から呼び出す
// ===== 描画 =====
// ===== ゲームループ =====
```

3. 得点の表示欄と、得点の変数

sample/index.html の <canvas> の上に、得点の表示欄を 1 行追加する。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>シューティングゲーム</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>SCORE: <span id="score">0</span></p>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

- 第 20 回と同じ形。SCORE: の後ろの数字だけを で囲む。
- 得点は Canvas には描かず、HTML の表示欄に出す。Canvas には、自機・弾・敵などの動くものだけを描く。

sample/main.js の「準備」と「ゲームの状態」の部分を、次のようにする。

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");
const scoreElement = document.getElementById("score");

// ===== ゲームの状態 =====
let score = 0;
let frameCount = 0;           // ゲーム開始からのフレーム数
```

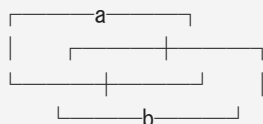
- scoreElement : 得点の表示欄 () を取り出して入れておく。第 20 回と同じ getElementById。

4. 四角形の当たり判定 (isHit)

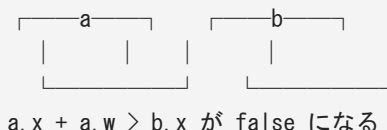
2つの四角形 a と b が重なっているかは、次の 4 つの条件で調べられる。4 つがすべて true のとき、重なっている。この 4 つの条件は、自分で考え出す必要はない。「この形で判定できる」と覚えて使う。

条件	意味
<code>a.x < b.x + b.w</code>	a の左端が、b の右端より左にある
<code>a.x + a.w > b.x</code>	a の右端が、b の左端より右にある
<code>a.y < b.y + b.h</code>	a の上端が、b の下端より上にある
<code>a.y + a.h > b.y</code>	a の下端が、b の上端より下にある

重なっている例



重なっていない例 (a が b より左にある)



`a.x + a.w > b.x` が `false` になる

- 1 つでも `false` になれば、横か縦のどちらかで離れているので、重なっていない。

「更新」の部分の `removeBullets` の下に、`isHit` を追加する。

```
// 2 つの四角形が重なっていたら true を返す
function isHit(a, b) {
  return a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y;
}
```

- 第 20 回で学んだ「`return` で値を返す関数」。ここでは、比較の結果 (`true` か `false`) を返している。このように、真偽値を返す関数には `is~` という名前を付けることが多い。
- 引数 `a` と `b` に、`x・y・w・h` を持つオブジェクトを渡す。弾も敵も自機も同じ形なので、どの組み合わせにも同じ `isHit` が使える。
- コンソールで試せる。`isHit({ x: 0, y: 0, w: 10, h: 10 }, { x: 5, y: 5, w: 10, h: 10 })` と入力すると `true`、2 つ目の `x` を 20 にすると `false` になる。

5. for の中の for (入れ子) と break

すべての弾と、すべての敵の組み合わせを調べるには、`for` の中に `for` を書く (`for` の入れ子)。

「更新」の部分の `isHit` の下に、`checkBulletHits` を追加する。

```
// 弾と敵の当たり判定
function checkBulletHits() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    for (let j = enemies.length - 1; j >= 0; j--) {
      if (isHit(bullets[i], enemies[j])) {
        bullets.splice(i, 1);
        enemies.splice(j, 1);
        score += 100;
        scoreElement.textContent = score;
        break; // この弾は消えたので、次の弾へ
      }
    }
  }
}
```

(1) for の入れ子

- 外側の `for (i)` で弾を 1 つ選び、内側の `for (j)` で、その弾とすべての敵を比べる。
- 内側と外側で、別の変数名 (`i` と `j`) を使う。
- 当たった弾と敵を `splice` で消すので、第 25 回と同じく、どちらも後ろから回す。

弾 ■	比べる敵 ■
弾 2	敵 1、敵 0
弾 1	敵 1、敵 0
弾 0	敵 1、敵 0

(弾 3 発・敵 2 体のとき)

(2) 当たったとき

- 弾と敵の両方を `splice` で消す (第 25 回と同じ書き方)。
- 得点を 100 増やし、`scoreElement.textContent = score;` で表示を書き換える (第 20 回と同じ書き方)。

(3) break

- `break;` : その `for` 文を、途中で終わらせる。`for` が入れ子のときは、`break` を書いた内側の `for` だけが終わる。
- 弾 `i` は、当たった時点で消えている。そのまま内側の `for` を続けると、消えた弾 (`bullets[i]`) を調べようとして、エラーになることがある。
- そこで `break` で内側の `for` を終わらせ、外側の `for` で次の弾へ進む。

「更新」の部分の `update` を、次のようにする。

```
function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
  updateEnemies();
  removeEnemies();
  removeBullets();
  checkBulletHits();
}
```

- 当たり判定は、全員を動かして、画面外のを消したあとに行う。
- 弾を当てると弾と敵が消え、画面の上の SCORE が 100 ずつ増える。

6. うまく動かないとき

症状	確認すること
当たっても消えない	<code>update</code> から <code>checkBulletHits</code> を呼んでいるか。 <code>isHit</code> の 4 つの条件の書き間違い
得点の表示が変わらない	<code>index.html</code> の <code>id="score"</code> と、 <code>getElementById("score")</code> の名前。 <code>scoreElement.textContent = score;</code> を書いたか
<code>Cannot read properties of undefined</code> というエラー	<code>break;</code> を書いたか。 <code>for</code> の条件 (<code>i >= 0</code> 、 <code>j >= 0</code>)
<code>Cannot set properties of null</code> というエラー	<code>getElementById</code> の名前と、HTML の <code>id</code> が同じか

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 当たり判定を作る

- 講義と同じように、 `isHit` と `checkBulletHits` を追加して、弾が敵に当たると、弾と敵が消えるようにする。
- `checkBulletHits` は、次の空欄（`_____`）を埋めて完成させる（得点の 2 行は演習 2 で書く）。

```
function checkBulletHits() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    for (let j = enemies.length - 1; j >= 0; j--) {
      if (isHit(_____, _____)) {
        bullets.splice(_____, 1);
        enemies.splice(_____, 1);
        break;
      }
    }
  }
}
```

演習 2 得点を HTML に表示する

- `exercise/index.html` に、講義と同じ得点の表示欄（`<p>SCORE: <span id="score">0</span></p>`）を追加する。
- `score` の変数と、表示欄を取り出す `scoreElement` を用意する。
- 敵を倒したら得点を増やし、表示を書き換える。1 体倒したときの得点は、自分で決める。
- 分からないときは、第 20 回の「ボタンを押すと得点が増える」を見直す。

発展（任意）

- `checkBulletHits` の `break;` の行の先頭に `//` を付けて、たくさん弾を撃ってみる。コンソールにエラーが出るかどうかを確かめる。確かめたら、`//` を消して元に戻す。

この回の終わりの `sample/main.js` 全体

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");
const scoreElement = document.getElementById("score");

// ===== ゲームの状態 =====
let score = 0;
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機・弾・敵 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
let bullets = [];
let enemies = [];
let shotTimer = 0;           // 次の弾を撃てるまでのフレーム数

// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;
let isSpacePressed = false;

document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
  if (event.key === " ") {
    isSpacePressed = false;
  }
});

// ===== 更新 =====
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
```

```

    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
    player.x = canvas.width - player.w;
  }
}

function updateBullets() {
  if (shotTimer > 0) {
    shotTimer -= 1;
  }
  // スペースキーを押していて、撃てる状態なら、弾を追加する
  if (isSpacePressed && shotTimer === 0) {
    bullets.push({ x: player.x + player.w / 2 - 2, y: player.y, w: 4, h: 12, speed: 8 });
    shotTimer = 10;
  }
  // すべての弾を上へ動かす
  for (let i = 0; i < bullets.length; i++) {
    bullets[i].y -= bullets[i].speed;
  }
}

function updateEnemies() {
  // 60 フレーム（約 1 秒）ごとに、敵を 1 体出す
  if (frameCount % 60 === 0) {
    enemies.push({ x: Math.floor(Math.random() * (canvas.width - 32)), y: -32, w: 32, h: 32, speed: 2 });
  }
  // すべての敵を下へ動かす
  for (let i = 0; i < enemies.length; i++) {
    enemies[i].y += enemies[i].speed;
  }
}

// 画面の下に出た敵を消す
function removeEnemies() {
  for (let i = enemies.length - 1; i >= 0; i--) {
    if (enemies[i].y > canvas.height) {
      enemies.splice(i, 1);
    }
  }
}

// 画面の上に出た弾を消す
function removeBullets() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    if (bullets[i].y + bullets[i].h < 0) {
      bullets.splice(i, 1);
    }
  }
}

// 2 つの四角形が重なっていたら true を返す
function isHit(a, b) {
  return a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y;
}

// 弾と敵の当たり判定

```

```

function checkBulletHits() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    for (let j = enemies.length - 1; j >= 0; j--) {
      if (isHit(bullets[i], enemies[j])) {
        bullets.splice(i, 1);
        enemies.splice(j, 1);
        score += 100;
        scoreElement.textContent = score;
        break; // この弾は消えたので、次の弾へ
      }
    }
  }
}

function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
  updateEnemies();
  removeEnemies();
  removeBullets();
  checkBulletHits();
}

// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function drawBullets() {
  ctx.fillStyle = "#ffff00";
  for (let i = 0; i < bullets.length; i++) {
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
  }
}

function drawEnemies() {
  ctx.fillStyle = "#ff4444";
  for (let i = 0; i < enemies.length; i++) {
    ctx.fillRect(enemies[i].x, enemies[i].y, enemies[i].w, enemies[i].h);
  }
}

function draw() {
  drawBackground();
  drawPlayer();
  drawBullets();
  drawEnemies();
}

// ===== ゲームループ =====

```

```
function gameLoop() {  
  update();  
  draw();  
  requestAnimationFrame(gameLoop);  
}  
  
gameLoop();
```